

4. DevOps Enviroments

Wednesday, July 15, 2026 4:32 PM

DevOps Environments

A **DevOps environment** is a **dedicated setup of infrastructure, software, and configuration** that supports specific phases of the software delivery lifecycle — from development and testing to production deployment.

Each environment is isolated but **replicates production in different degrees**, allowing the team to build, test, and deploy confidently.

Why Multiple Environments?

Using multiple environments allows:

- Safe development without affecting live users
- Thorough testing in near-production conditions
- Controlled and phased rollout of features
- Easier debugging and rollback
- Enforcement of quality gates (like in CI/CD)

Common Types of DevOps Environments

1. Local Development Environment

Where developers write and test code on their own machines.

Tools:

- **IDE:** VS Code, IntelliJ
- **Containerization:** Docker / Docker Compose
- **Minikube / Kind** for local Kubernetes
- **Mock APIs** and **SQLite/local DBs**

Used For:

- Writing and debugging code
- Running unit tests
- Experimenting with new features

Example:

A Java developer runs `mvn spring-boot:run` to locally test a new login API before pushing code.

◆ 2. Development Environment (DEV)

First shared environment where features are deployed after passing CI.

🔧 Tools:

- Kubernetes Cluster (Namespace: dev)
- Jenkins CI/CD or GitHub Actions
- Docker Registry for images
- Prometheus/Grafana for monitoring

✅ Used For:

- Integrating new features
- Team collaboration
- Smoke testing
- Basic integration testing

🧠 Characteristics:

- Often unstable
- Frequent redeployments
- Uses mock data / dev secrets
- Debug logs enabled
- Feature flags for in-progress code

🔴 Example:

After CI pipeline completes, the Docker image is deployed to the dev namespace using:

```
kubectl apply -f dev-deployment.yaml
```

◆ 3. QA / Testing Environment

Used by testers for validating application quality.

🔧 Tools:

- Selenium, JMeter, Postman for test automation
- TestNG/JUnit results in Jenkins
- Test data populated by scripts or synthetic generation

✅ Used For:

- Functional testing
- Regression testing
- API testing
- Performance/load testing

🧠 Characteristics:

- Test users and test DBs
- More stable than Dev
- Periodically reset

🔴 Example:

Testers verify that a bug fix doesn't reintroduce a login failure by rerunning automated regression tests.

◆ 4. Pre-Production / PPD / UAT Environment

Mimics production closely — a **mirror of the production stack**.

🔧 Tools:

...

🔴 Example:

Testers verify that a bug fix doesn't reintroduce a login failure by rerunning automated regression tests.

◆ 4. Pre-Production / PPD / UAT Environment

Mimics production closely — a **mirror of the production stack**.

🔧 Tools:

- Kubernetes Cluster (Namespace: ppd)
- Real secrets from Vault/SealedSecrets
- DNS & SSL setup identical to prod
- Canary deployment support

✅ Used For:

- Final validation before prod
- Business user approval (UAT)
- Load/stress testing with production-like data

🧠 Characteristics:

- Ingress + HTTPS configured
- Connected to staging backend systems
- Metrics + Logging fully enabled

🔴 Example:

Before releasing a new invoice system, business users test it in PPD with real workflows and sign off.

◆ 5. Production (PROD) Environment

This is the **live environment** accessed by real users.

🔧 Tools:

- Kubernetes/EC2/VMs for infrastructure
- Helm/ArgoCD/Flux for GitOps deployments
- cert-manager for TLS
- Prometheus/Grafana for live metrics
- Loki or ELK for logging
- Sentry/NewRelic for error tracking

✅ Used For:

- Running the actual app for customers
- Ensuring high availability and performance
- Capturing real-time logs and metrics

🧠 Characteristics:

- Immutable deployments (tagged versions)
- Load-balanced, auto-scaled
- Restricted access

-
- 24/7 Monitoring & alerting
 - Rollback enabled
 - Strong RBAC & security policies

🔴 Example:

When a developer merges code to the main branch, Jenkins deploys the new Docker image using Helm:

- 24/7 Monitoring & alerting
- Rollback enabled
- Strong RBAC & security policies

🔴 **Example:**

When a developer merges code to the main branch, Jenkins deploys the new Docker image using Helm:

```
helm upgrade --install app-prod ./charts --values values-prod.yaml
```

♦ 6. Disaster Recovery (DR) Environment

A backup clone of PROD, used during critical outages.

🔧 **Tools:**

- Same as PROD but on different region/cloud
- Replicated DB (cold/hot)
- Automated failover mechanism

✅ **Used For:**

- Business continuity during failures
- Simulated DR drills

🧠 Additional Types

Environment	Purpose
Sandbox	Experimental playground
Demo	Demos for clients or internal
Canary	Release new features to subset
Blue/Green	Parallel deployment strategy

🌐 Example Environment Flow in a Real DevOps Lifecycle

1. Dev pushes code → DEV
2. QA runs tests in QA
3. Business reviews in PPD
4. Code gets promoted to PROD
5. If PROD fails, rollback or shift traffic to Blue-Green/DR

✅ Best Practices for Managing Environments

- Use **separate namespaces** or **clusters** for each environment
- **Parameterize deployments** using Helm or Kustomize
- Automate deployments using **GitOps** tools like ArgoCD
- Inject secrets securely via **Vault/SealedSecrets**
- Monitor and alert every environment (not just prod)

✅ Best Practices for Managing Environments

- Use **separate namespaces** or **clusters** for each environment
 - **Parameterize deployments** using Helm or Kustomize
 - Automate deployments using **GitOps** tools like ArgoCD
 - Inject secrets securely via **Vault/SealedSecrets**
 - Monitor and alert every environment (not just prod)
 - Log retention policies should vary by environment
 - Lock PROD deployments with **manual approvals**
 - Automate **environment creation for each PR**
-

📊 Tool Stack per Environment (Sample Matrix)

Tool	Dev	QA	PPD	Prod
Jenkins	✅	✅	✅	✅
Docker	✅	✅	✅	✅
Kubernetes	✅	✅	✅	✅
SonarQube	✅	✅	✅	❌
Trivy	✅	✅	✅	✅
Prometheus/Grafana	✅	✅	✅	✅
HashiCorp Vault	❌	✅	✅	✅
cert-manager	❌	✅	✅	✅
ArgoCD	✅	✅	✅	✅